

PAIVO PAYMENTS

Payment Orchestration Platform Modernization

Enterprise Architecture Definition Document

A TOGAF Architecture Development Method (ADM) Case Study

Preliminary Phase through Phase H — Full ADM Cycle

Prepared by: Enterprise Architecture Function

Document Reference: PVO-EA-POP-001 | Version 1.0

August 2026

FICTIONAL CASE STUDY — Paivo Payments, all named individuals, and all figures in this document are invented for demonstration and portfolio purposes. This document does not describe any real payments company or PSP, and no figures herein should be relied upon as real-world data.

Table of Contents

1. Executive Summary.....	4
2. Preliminary Phase — Governance & Principles.....	5
2.1 Architecture Principles.....	5
2.2 Governance Framework Setup.....	5
3. Phase A — Architecture Vision & Scope.....	7
3.1 Problem Statement & Target-State Vision.....	7
3.2 Scope Boundaries.....	7
3.3 Success Metrics.....	8
3.4 Stakeholder Map.....	8
3.5 Business Case Summary.....	9
4. Phase B — Business Architecture.....	11
4.1 As-Is Business Architecture.....	11
4.2 To-Be Business Architecture.....	11
4.3 Business Capability Map.....	12
4.4 Business Footprint.....	13
5. Phase C — Information Systems Architecture.....	15
5.1 Application Architecture.....	15
5.2 Data Architecture.....	16
5.3 Data Ownership / Domains.....	17
6. Phase D — Technology Architecture.....	19
6.1 Reference Architecture Pattern.....	19
6.2 Technology Standards.....	19
7. Phase E — Opportunities & Solutions.....	21
7.1 Gap Analysis.....	21
7.2 Solution Building Blocks.....	21
7.3 Vendor Evaluation.....	22
8. Phase F — Migration Planning.....	23
8.1 Migration Roadmap.....	23
8.2 Transition Architecture — Dual-Rail Coexistence.....	23
9. Phase G — Implementation Governance.....	25
9.1 Non-Compliance Handling.....	25
9.2 Architecture Contracts.....	25
10. Phase H — Change Management.....	26

11. Architecture Decision Records.....27

Appendix A — Diagram Catalog Index.....28

Appendix B — Disclaimer.....29

1. Executive Summary

Paivo Payments connects merchants to payment rails — UPI, cards, wallets, net banking — through integrations built point-to-point: one bespoke integration per merchant, per rail, often per PSP. That model is now failing Paivo on three fronts at once. Every new rail partnership costs roughly \$180K to integrate regardless of the volume it carries, so unit economics never improve with scale. Reconciliation runs separately per rail with no single source of truth, so settlement discrepancies surface late, in Finance's monthly close, rather than in near-real time. And the regulator's ISO 20022 structured-messaging mandate cannot be met under the current model without retrofitting all 20 existing integrations individually — a \$1.3M, high-risk undertaking with no shared path to compliance.

The Enterprise Architecture function recommends building a central payment orchestration hub: one platform every merchant and rail connects through, backed by a single ledger of truth for reconciliation and one shared layer for ISO 20022 compliance. Rail-specific complexity — PSP APIs, protocol quirks — is isolated into small, independently replaceable adapters, so a PSP changing its API touches one component, not the whole platform. A point-to-point mesh improved with shared libraries and a hybrid shared-ledger-only design were both evaluated and rejected (ADR-001): neither closes the idempotency gap that has already produced two documented double-debit incidents in eighteen months.

The program is estimated at \$17.52M in total cost over three years — against \$8.40M for continuing as-is, which itself includes a forced \$1.3M ISO 20022 retrofit that happens regardless of which path Paivo takes. The gap is front-loaded: Year 1 build cost is the majority of the delta, and it inverts by Year 3, where the orchestration platform is already running \$871K cheaper per year than the as-is model. Cumulative payback is projected at approximately Year 6–7 of platform life, the normal timeline for infrastructure re-platforming evaluated against a 7-10 year operating life, not a short-term ROI bar.

The migration runs across six waves over roughly eighteen months of live rail cutovers (following a Wave 0 foundation build), sequenced by risk rather than sales priority — the lowest-volume rail (net banking) migrates first so early production issues are discovered against the smallest possible blast radius. The ISO 20022 transformation layer is built and validated in Wave 0, well ahead of the regulatory deadline, so compliance readiness does not depend on the full multi-wave migration finishing on time. This document is the consolidated Architecture Definition Document for the program, compiled across every phase of the TOGAF Architecture Development Method — Preliminary Phase governance setup through Phase H change management — and is intended as a single, self-contained reference of the architecture, its rationale, and its governance.

2. Preliminary Phase — Governance & Principles

Before Phase A vision work began, Paivo's architecture function stood up a governance model sized specifically for the program's scale — large enough to give the orchestration program real teeth against a regulatory deadline, small enough not to become the bottleneck that point-to-point delivery teams could use as a reason to route around architecture entirely.

2.1 Architecture Principles

Twelve architecture principles were ratified by the Architecture Review Board (ARB) and govern every subsequent decision in this program. Each ADR and architecture contract in this document cites these principles by number.

#	Principle	Statement (Condensed)
1	Single Ledger of Truth	Every payment transaction, regardless of rail or PSP, is recorded exactly once in a single, authoritative ledger service that all downstream reconciliation, settlement, and reporting consumes.
2	Rail-Agnostic Orchestration Core	The orchestration core (routing, retries, idempotency, ledger writes) contains no rail-specific or PSP-specific logic; all such logic is isolated to adapters.
3	Regulatory Compliance Is Non-Negotiable Scope, Not a Feature	ISO 20022 messaging, PCI-DSS scope isolation, and audit trail requirements are mandatory constraints on every design, not backlog items that compete with feature work.
4	Idempotency by Design, Not by Convention	Every state-changing operation must be idempotent at the API contract level, platform-enforced, not left to caller discipline.
5	Data Minimization at the Boundary	Sensitive cardholder and account data is tokenized or truncated at the earliest point of ingestion and never persisted in full form outside the PCI-DSS-scoped vault.
6	Build the Transformation Layer Once, Centrally	ISO 20022 (and any future message standard) transformation logic lives in one shared layer, not duplicated per adapter.
7	Design for Multi-Rail Extensibility, Not Multi-Rail Prediction	Adding an unspecified future rail must be a bounded, well-defined task, without pre-building support for rails not yet on the roadmap.
8	Availability Commitments Are Explicit and Tiered	Every service carries an explicit availability tier (Tier-0 through Tier-2) with a stated SLA; architecture decisions state which tier they are designed to meet.
9	Vendor and Rail Substitutability	No single PSP, rail, or orchestration vendor product may become structurally load-bearing to the point that switching it requires a core platform redesign.
10	Traceability from Business Requirement to Deployed Component	Every solution building block deployed under this program must trace back to a stated business or regulatory requirement.
11	Prefer Managed Infrastructure Over Custom-Built Undifferentiated Capability	Where a managed cloud service or reputable commercial product meets a non-differentiating requirement, it is preferred over a custom-built equivalent.
12	Architecture Decisions Are Recorded, Not Tribal	Any decision that is expensive to reverse, affects more than one team, or trades off a stated principle must be captured as a formal ADR and reviewed

#	Principle	Statement (Condensed)
		by the ARB.

2.2 Governance Framework Setup

The ARB has five voting members: the Chief Architect (chair, tie-break vote), the Head of Platform Engineering, the Head of Risk & Compliance (regulatory sign-off authority), the Head of Payments Product, and the Principal Security Architect. Standing non-voting invitees attend by topic: engineering leads for the domain under review, a Finance representative for cost-impacting decisions, and an external auditor liaison invited quarterly and for any ISO 20022 or PCI-DSS-scoped decision. A larger, nine-person cross-functional board modeled on Paivo's legacy change advisory board was considered and rejected — quorum logistics were already the single biggest complaint about that legacy process, adding a median nine calendar days to any decision. A two- or three-person board was also rejected as too narrow to carry regulatory and security sign-off without a separate escalation step for the majority of decisions.

The ARB operates on four cadences: a weekly, 60-minute standing session with decisions logged the same day; Architecture Contract review on submission, with a five-business-day target turnaround; a monthly principle-compliance self-audit; and a quarterly deep review of the full program against the Phase F roadmap, with the external auditor liaison present given the ISO 20022 deadline. Delivery teams retain full decision rights over implementation choices that do not touch a stated principle or cross a service boundary — deliberately, to keep architecture from becoming a bottleneck for routine engineering. Any decision introducing a new solution building block, changing a multi-team interface, touching PCI-DSS scope, or deviating from an approved technology standard requires a submitted Architecture Contract. A waiver process allows a time-boxed exception (default two quarters) to any principle, voted by the ARB and re-reviewed rather than granted permanently. The Head of Risk & Compliance additionally holds a standing veto, exercisable without a full ARB vote and ratified at the next standing session, over any decision that would put the ISO 20022 or PCI-DSS deadline at risk.

Disagreement between a delivery team and the ARB escalates to the Executive Sponsor (the CTO) with both positions documented — invoked once in the program's first two quarters, over the orchestration hub vs. mesh topology decision recorded in ADR-001. Every Architecture Contract template includes a mandatory "principle impact" section requiring the submitting team to state explicitly which principles a proposal touches and how; a submission silent on this section is returned unreviewed. This single procedural gate is credited with catching the majority of principle violations before they reach a vote, since reviewers evaluate a stated trade-off rather than discover an unstated one.

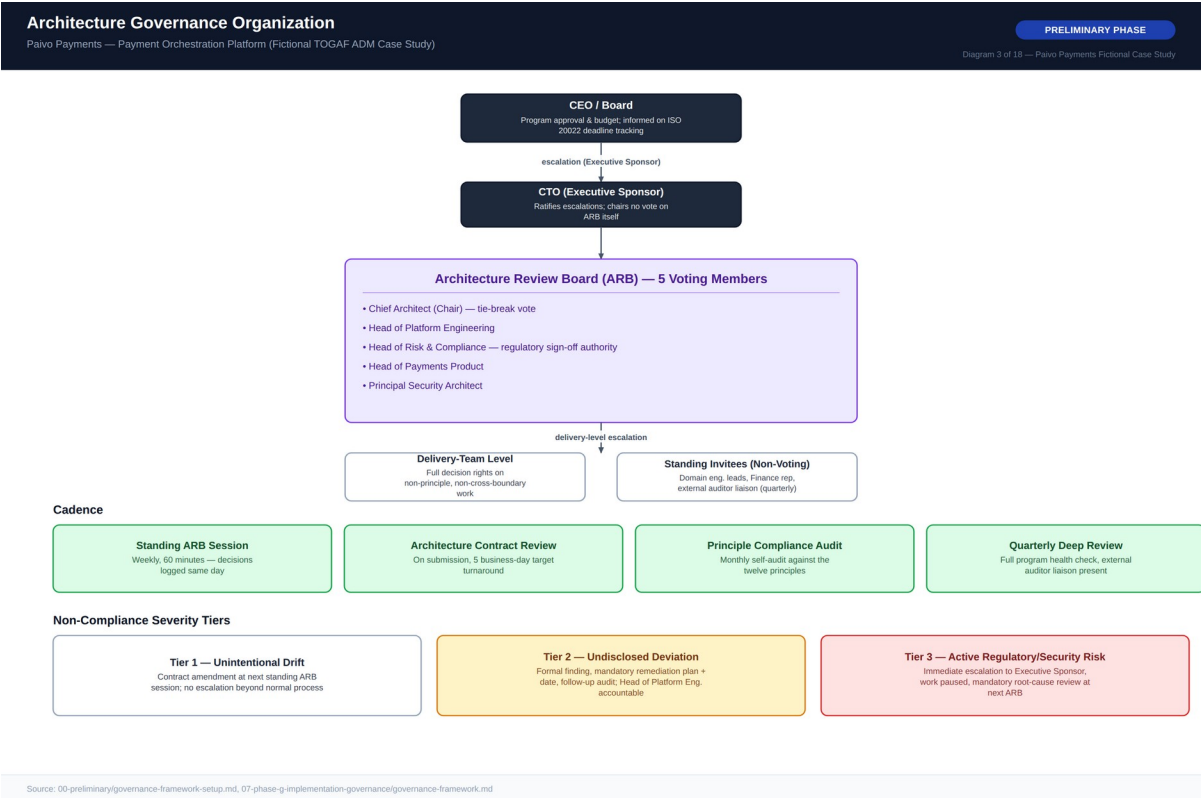


Diagram 3 — Architecture Governance Organization

3. Phase A — Architecture Vision & Scope

3.1 Problem Statement & Target-State Vision

Paivo's current architecture is a point-to-point mesh: every merchant-rail combination has its own bespoke integration, built independently by whichever team owned that integration at the time. This model scaled acceptably while Paivo operated a small number of rails in a single market; it now fails on three fronts simultaneously. Integration cost does not amortize — each new PSP partnership or rail expansion requires bespoke work regardless of the volume it will carry. Reconciliation is fragmented — each rail integration reconciles independently against its own PSP statement format, so settlement discrepancies surface late in month-end close rather than near-real time. And the architecture cannot meet the ISO 20022 mandate as-is — compliance under the current model means building message transformation logic independently into every one of the twenty existing integrations, a cost that scales with integration count, not with the size of the problem.

The target state is a central payment orchestration hub that all merchants and rails connect through once, replacing $N \times M$ point-to-point integrations. The hub owns transaction routing, retries, idempotency, and writes every transaction to a single ledger service. Rail- and PSP-specific protocol differences are isolated to individually replaceable adapters, with a shared message transformation layer between adapters and the orchestration core translating rail-native formats to and from a canonical internal schema — and, where required, to ISO 20022 structured messages — built once, not per adapter. This directly resolves all three problem drivers: onboarding a new rail becomes a bounded adapter-build exercise, reconciliation runs against one ledger instead of N fragmented processes, and ISO 20022 compliance is achieved by extending the shared transformation layer once.

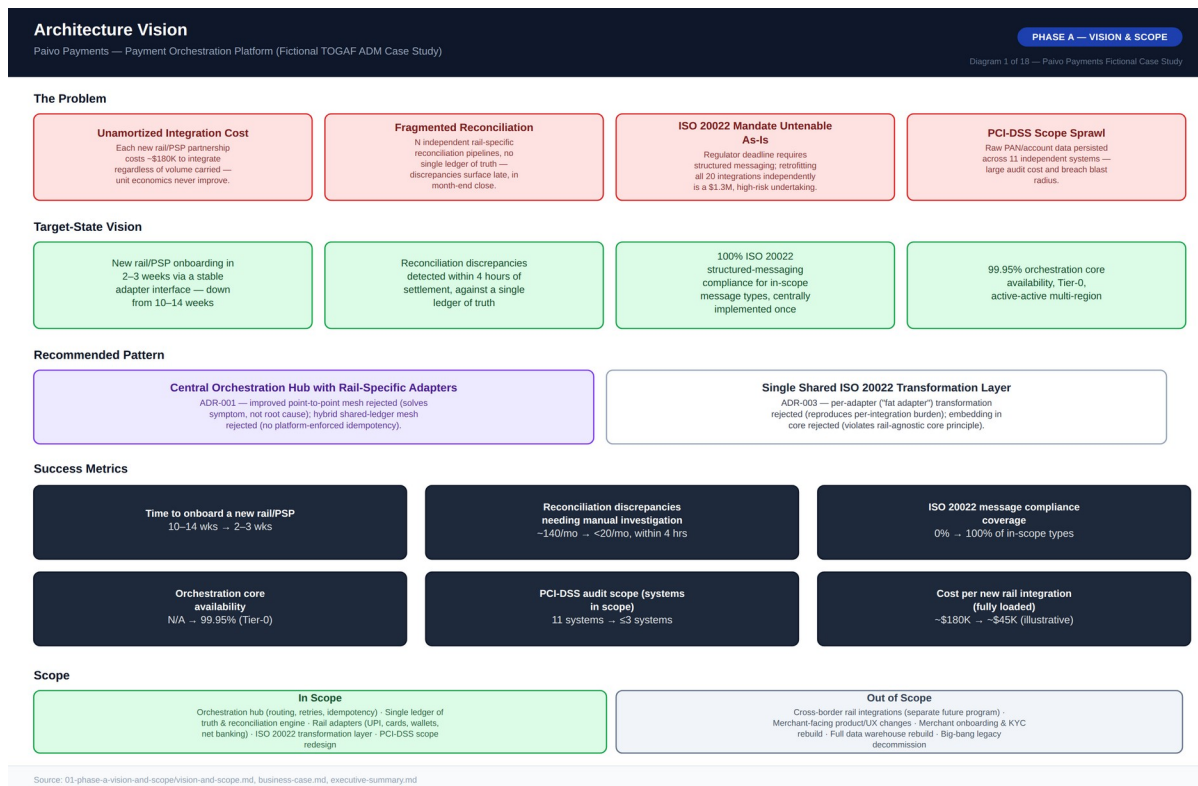


Diagram 1 — Architecture Vision One-Pager

3.2 Scope Boundaries

In scope: design and build of the central orchestration hub (routing, retries, idempotency, orchestration state management); the single ledger of truth service and its reconciliation engine; rail adapters for Paivo's current live rails — UPI, card networks, major wallets, and net banking; the ISO 20022 message transformation layer, scoped to payment initiation and payment status reporting; migration of existing merchant traffic from point-to-point integrations to the orchestration hub, phased by rail and merchant tier; and PCI-DSS scope redesign as a consequence of consolidating card data handling into the hub.

Explicitly out of scope, each evaluated and excluded to keep the program focused on the regulatory deadline: cross-border payment rail integrations themselves — the hub is designed to accommodate a future cross-border adapter, but building the actual rail connections is a separate, subsequent program, since coupling a hard regulatory deadline to a much less certain business initiative was assessed as an unacceptable schedule risk; merchant-facing product changes (checkout UX, dashboard redesign, pricing), which have their own roadmap and owner; replacement of Paivo's core merchant onboarding and KYC system, since it does not sit in the payment execution path this program re-architects; full data warehouse/analytics platform rebuild, since the ledger will emit events to the existing analytics pipeline without rebuilding it; and decommissioning legacy point-to-point integrations on day one — a big-bang cutover was considered and rejected as too high-risk for a business processing live merchant funds.

3.3 Success Metrics

Metric	Baseline (As-Is)	Target (18 Months Post-Launch)
Time to onboard a new rail/PSP integration	10–14 weeks	2–3 weeks
Reconciliation discrepancies requiring manual investigation	~140/month across rails	<20/month, detected within 4 hours of settlement
ISO 20022 message compliance coverage	0% (no structured messaging)	100% of in-scope message types by regulatory deadline
Orchestration core availability	N/A (no shared core exists)	99.95% (Tier-0 SLA)
PCI-DSS audit scope (systems in scope)	11 systems store or touch raw PAN	≤3 systems
Cost per new rail integration (fully loaded)	~\$180K per rail	~\$45K per rail

3.4 Stakeholder Map

Eleven stakeholder groups were assessed on power and interest to determine engagement strategy, from the CEO/Board (Accountable for program approval and budget) and CTO as Executive Sponsor (Accountable for target-state architecture) through the externally facing PSP/Rail Partners and External Auditor, to the Existing Point-to-Point Integration Engineers whose legacy code is being retired. A full RACI matrix governs program-level decisions: the Head of Risk & Compliance is Accountable/Responsible for ISO 20022 scope and

compliance sign-off, the Principal Security Architect and Head of Platform Engineering share Responsible/Accountable for PCI-DSS scope redesign, and the Head of Payments Product is Accountable/Responsible for merchant communication during migration.

Two stakeholder tensions required explicit ARB resolution rather than being smoothed over. Platform Engineering wanted a slower, more thoroughly tested migration, while Payments Product wanted rail coverage parity as fast as possible to support sales commitments already made to prospective enterprise merchants — resolved by sequencing the migration roadmap by risk (lowest-volume rail first) rather than sales priority, with an explicit executive-approved exception process. The Security Architect wanted all PSP adapters built in-house for full code-level audit control, while Platform Engineering wanted to evaluate commercial orchestration middleware to save build time against the regulatory deadline — resolved via the vendor evaluation, which scored options partly on auditability, not just cost and speed.

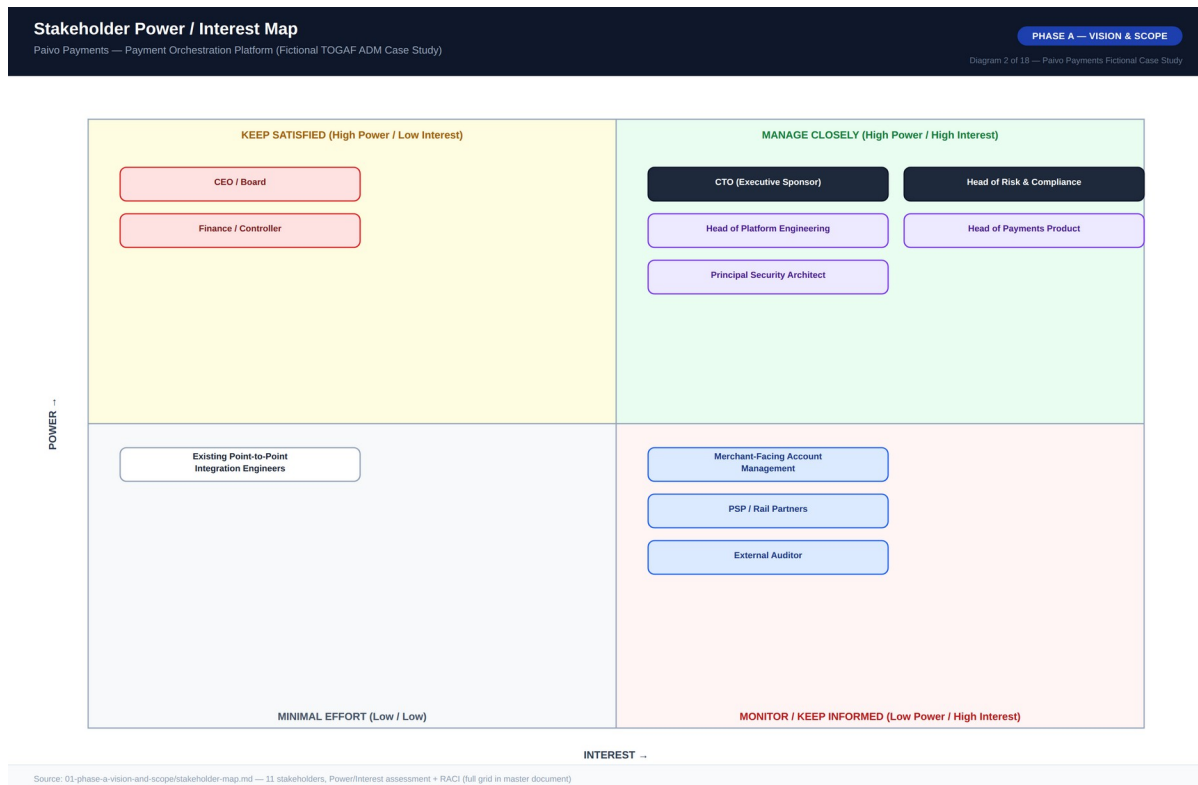


Diagram 2 — Stakeholder Power / Interest Map

3.5 Business Case Summary

The business case totals \$17.52M over the three-year program — dominated by the \$4.62M orchestration core and ledger build and the \$5.61M adapter build for all fourteen existing rails — against \$8.40M for the as-is model, which itself carries a forced \$1.3M ISO 20022 retrofit in Year 2 regardless of which path is taken. Year-over-year, the to-be model is \$8.13M more expensive in Year 1 and \$1.86M more expensive in Year 2, then \$871K cheaper in Year 3 — a \$9.12M cumulative extra spend through Year 3, evaluated against a 7–10 year platform lifespan rather than a 3-year ROI bar. Cumulative payback is projected at approximately Year 6–7 of platform life. Full arithmetic, the cost-breakdown line items, and the bar-chart comparison appear in Diagram 18.

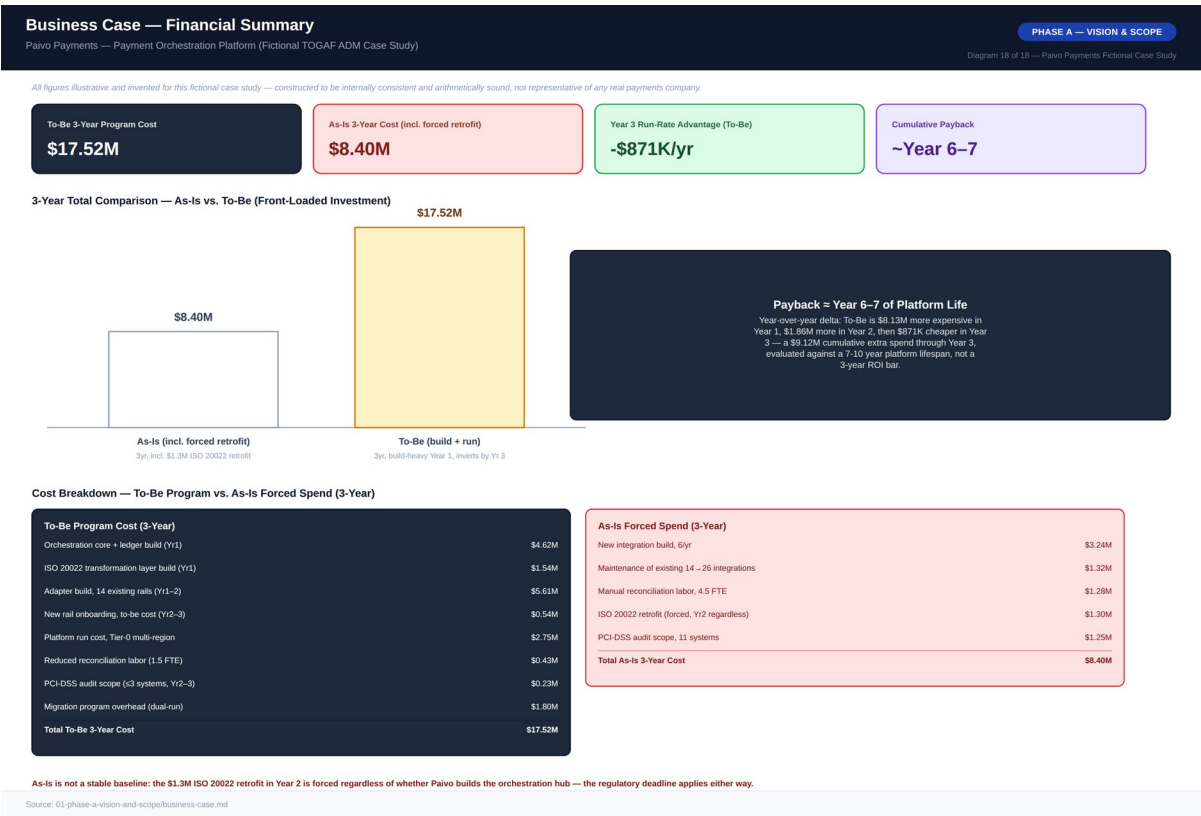


Diagram 18 — Business Case Financial Summary

4. Phase B — Business Architecture

4.1 As-Is Business Architecture

Paivo's payment execution business process today is organized around the rail, not the merchant or the transaction. When a merchant wants to accept a new payment method, Paivo's integration team scopes and builds a dedicated point-to-point connection to that rail's PSP: authentication, request/response mapping, webhook handling, and a bespoke reconciliation job pulling the PSP's settlement file and matching it against Paivo's own transaction log for that rail only. Payment initiation, status tracking, and reconciliation all run as N parallel, independently-maintained pipelines rather than one process with N pluggable rail connections. Each pipeline has its own retry logic and its own idempotency handling — inconsistently implemented, with some rails de-duplicating on merchant order ID and others not de-duplicating at all, relying on manual finance review to catch double-charges.

Reconciliation sits organizationally in Finance Operations, not Engineering: 4.5 FTEs each own a subset of rails, manually cross-referencing PSP settlement files against internal transaction records in spreadsheets and semi-automated scripts built incrementally over several years. There is no single authoritative transaction record — the closest approximation is the union of four-plus rail-specific databases, and discrepancies between them are the norm, discovered largely during month-end close. Compliance reporting is similarly fragmented: with no shared message transformation capability, any regulator-mandated structured messaging format has historically been handled by asking each rail integration owner to add ad hoc formatting logic to their own pipeline — precisely the pattern that makes the ISO 20022 mandate untenable under the current model.

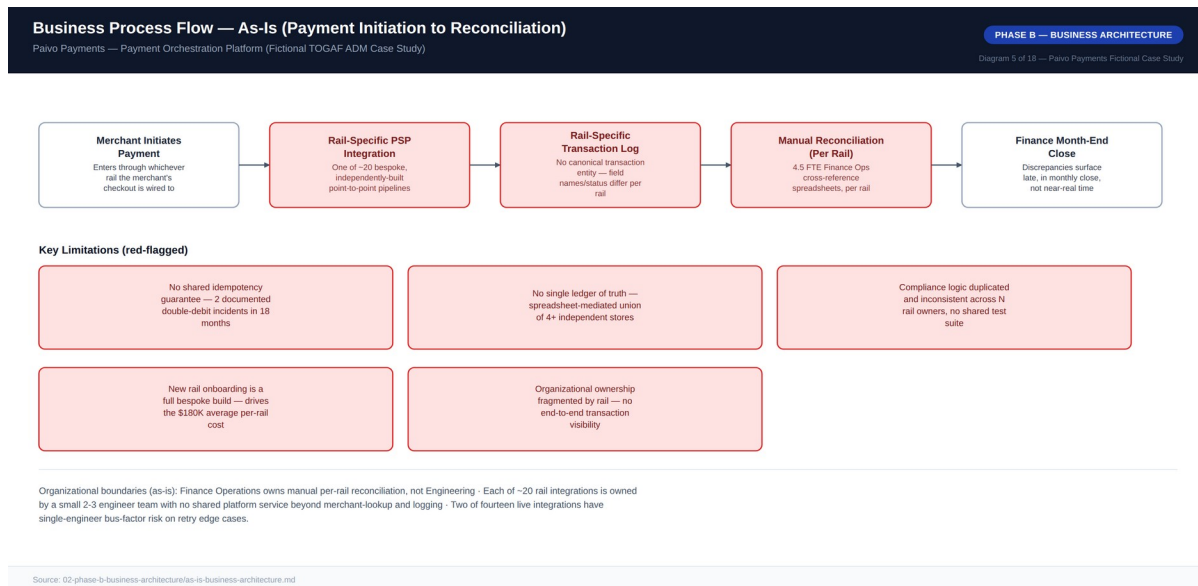


Diagram 5 — Business Process Flow, As-Is (Payment Initiation to Reconciliation)

4.2 To-Be Business Architecture

In the target state, payment initiation, status tracking, reconciliation, and compliance messaging are organized around one orchestration process with pluggable rail adapters. A merchant's payment request enters the orchestration hub once; the hub determines routing and delegates protocol-specific work to the relevant adapter, but retry policy, idempotency enforcement, and the transaction's authoritative record all live in the shared orchestration core regardless of rail. Every transaction is written once to the single ledger service as its terminal source of truth, and reconciliation becomes a single continuous process matching normalized settlement files against the ledger — the direct mechanism by which discrepancy detection moves from month-end close to within hours of settlement.

ISO 20022 is handled by the shared transformation layer between adapters and the orchestration core, converting the canonical internal representation to and from whatever wire format a rail or regulator requires, in one place, tested once. Bringing a new rail online becomes: build an adapter that speaks the rail's native protocol and emits the canonical internal event — the core, ledger, reconciliation, and compliance messaging require no rail-specific changes. Organizationally, this shifts reconciliation from a Finance Operations function assembling data from Engineering-owned silos, to a shared responsibility where Finance Operations consumes a single trustworthy ledger view and Engineering owns the platform that guarantees its accuracy. What deliberately stays the same: dispute and chargeback handling is already mature (3/5) and held steady, not re-architected — investment goes where the maturity gap is largest.

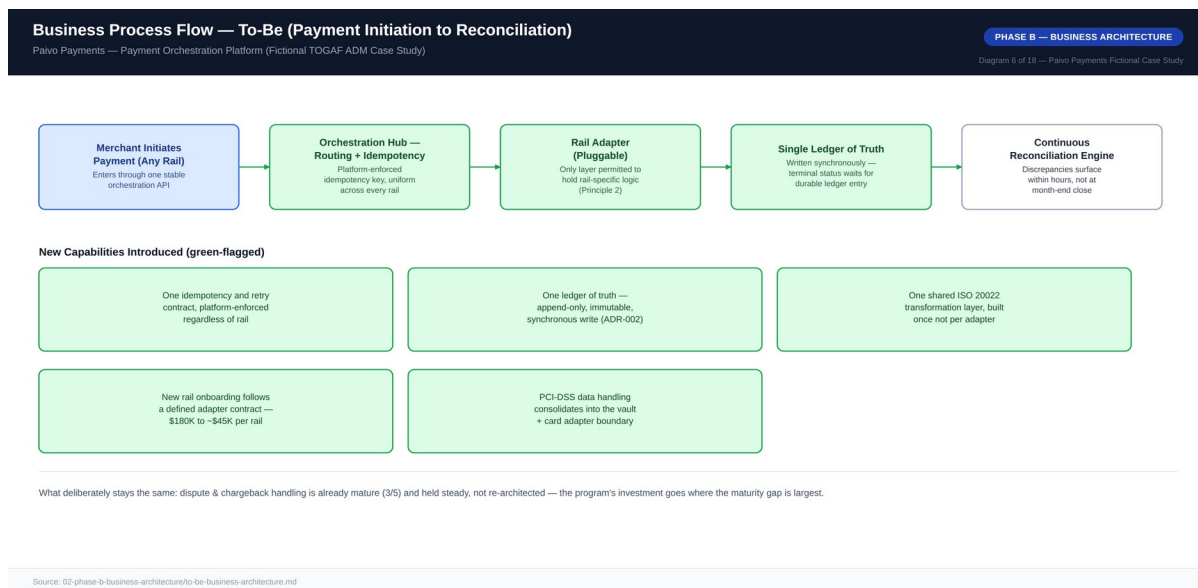
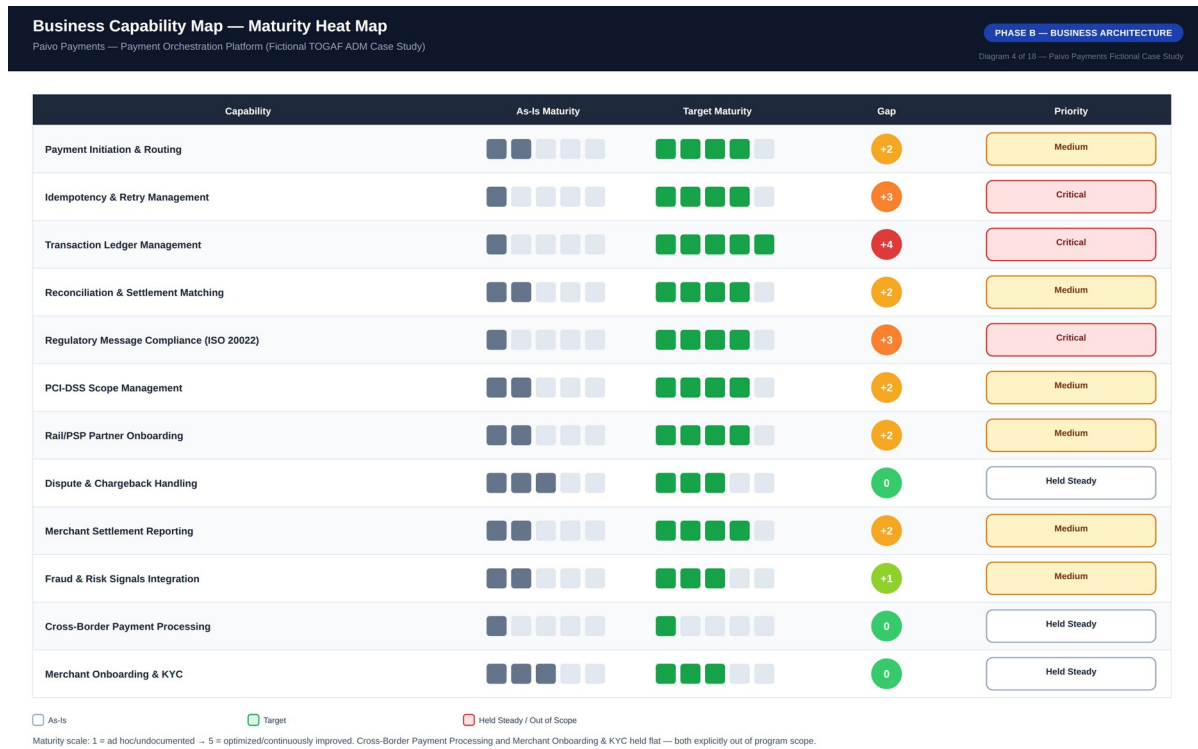


Diagram 6 — Business Process Flow, To-Be (Payment Initiation to Reconciliation)

4.3 Business Capability Map

Twelve business capabilities were rated jointly by the ARB and capability owners on a 1–5 maturity scale, as-is versus target. Transaction Ledger Management carries the single largest gap (1→5), reflecting that a real ledger does not meaningfully exist today — fragmented rail-specific records are not a ledger — and is the capability the entire program is structurally organized around. Idempotency & Retry Management and Regulatory Message Compliance (ISO 20022) both carry a gap of 3, and together with the ledger gap anchor

Wave 0 and Wave 1 of the migration roadmap. Two capabilities are deliberately held flat rather than improved: Cross-Border Payment Processing and Merchant Onboarding & KYC, both explicitly excluded from program scope. Dispute & Chargeback Handling is rated adequately mature already (3/5) and was deliberately not prioritized for uplift, expected to benefit indirectly from the ledger and reconciliation improvements without direct investment.



Source: 02 phase-b-business-architecture/capability-map.mxd — 12 capabilities rated jointly by ARB and capability owners

Diagram 4 — Business Capability Map, Maturity Heat Map

4.4 Business Footprint

The as-is application landscape is dominated by roughly twenty independently deployed integration services, each owning the full vertical stack for one rail/PSP combination: API client, authentication, request/response mapping, webhook receiver, local persistence, and a reconciliation batch job. There is no shared library or platform service between them beyond a common merchant-lookup API and a shared logging/monitoring baseline. Each integration is typically owned by a small two-to-three-engineer team, and institutional knowledge lives largely with that team — two of the fourteen live integrations currently have a single engineer who understands their retry edge cases in depth, a real bus-factor risk. Merchant-facing checkout and dashboard applications call directly into whichever integration service matches the merchant's selected rail, meaning the client-facing surface has direct knowledge of which backend service handles which rail, a coupling that also complicates adding new rails since client applications need updates alongside backend integration work.

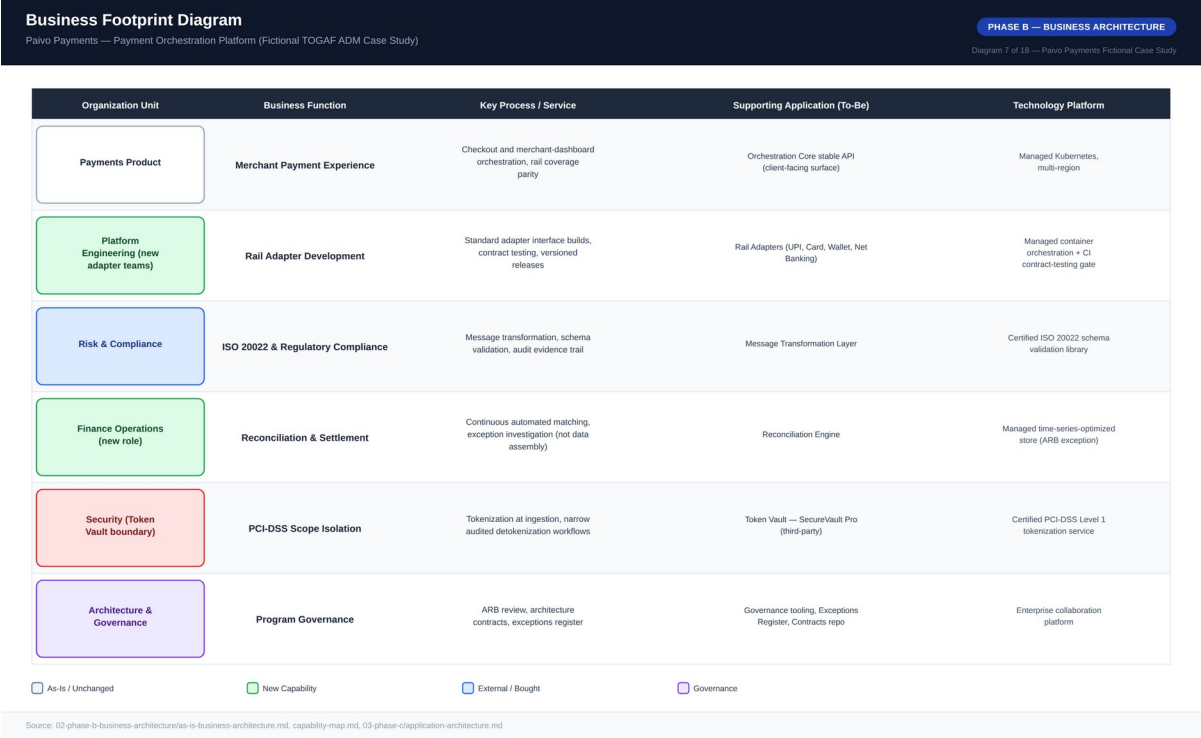


Diagram 7 — Business Footprint Diagram

5. Phase C — Information Systems Architecture

5.1 Application Architecture

The as-is landscape, shown in Diagram 8, has no API gateway and no shared adapter contract — every rail connection is genuinely point-to-point, the direct cause of the linear integration-maintenance cost growth this program exists to eliminate.

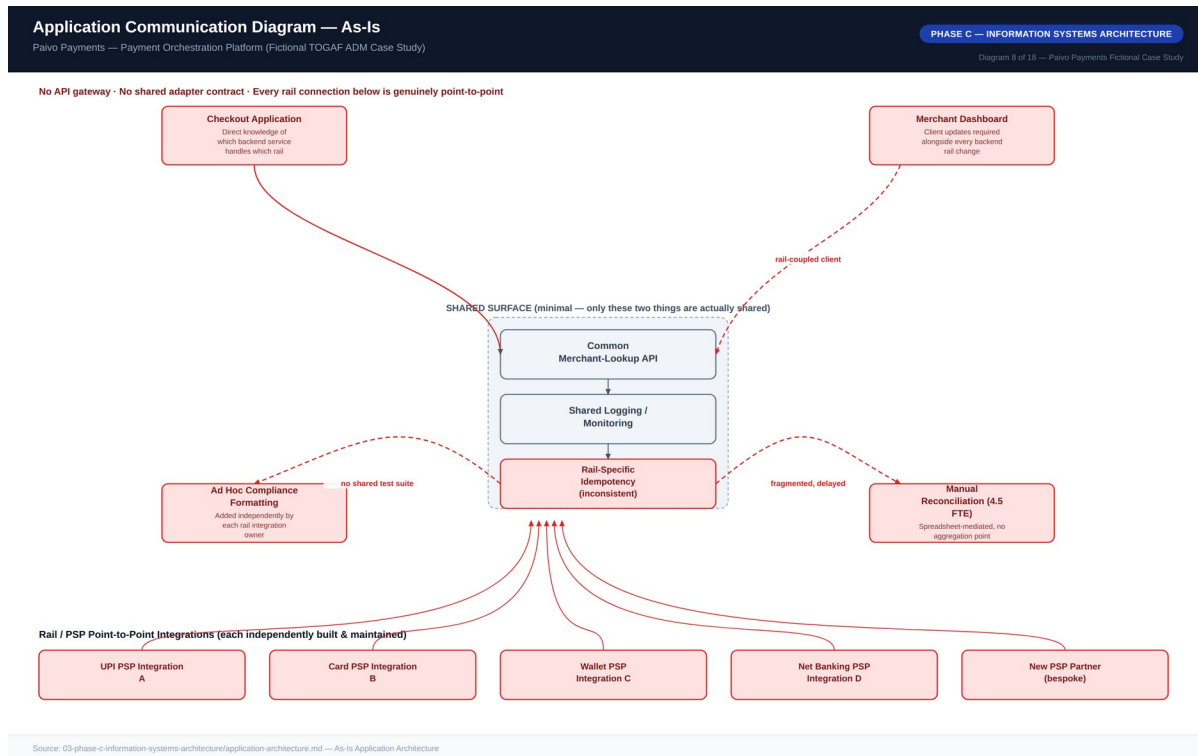


Diagram 8 — Application Communication Diagram, As-Is

The to-be architecture separates concerns into four independently deployable and independently scalable layers. The Orchestration Core is the single service, deployed active-active multi-region per architecture principle 8, owning routing, idempotency enforcement, retry policy, and ledger writes, exposing one stable API to every caller regardless of which rail ultimately processes the payment. The Message Transformation Layer sits between the orchestration core and adapters, converting the canonical internal schema to and from rail-native and ISO 20022 formats, deployed as a shared, versioned service rather than embedded logic (ADR-003). Rail Adapters — one per rail/PSP — are the only layer permitted to contain rail-specific logic, per architecture principle 2. The Ledger & Reconciliation Services keep the ledger as sole writer of authoritative transaction state, with reconciliation as a separate, read-only consumer, structurally distinct so a bug in reconciliation logic can never corrupt ledger state. A "fat adapter" pattern embedding transformation logic in each adapter was seriously considered — it would have shortened any single adapter's build time by an estimated 15–20% — but was rejected because it reproduces, one layer down, exactly the point-to-point maintenance burden the program exists to eliminate.

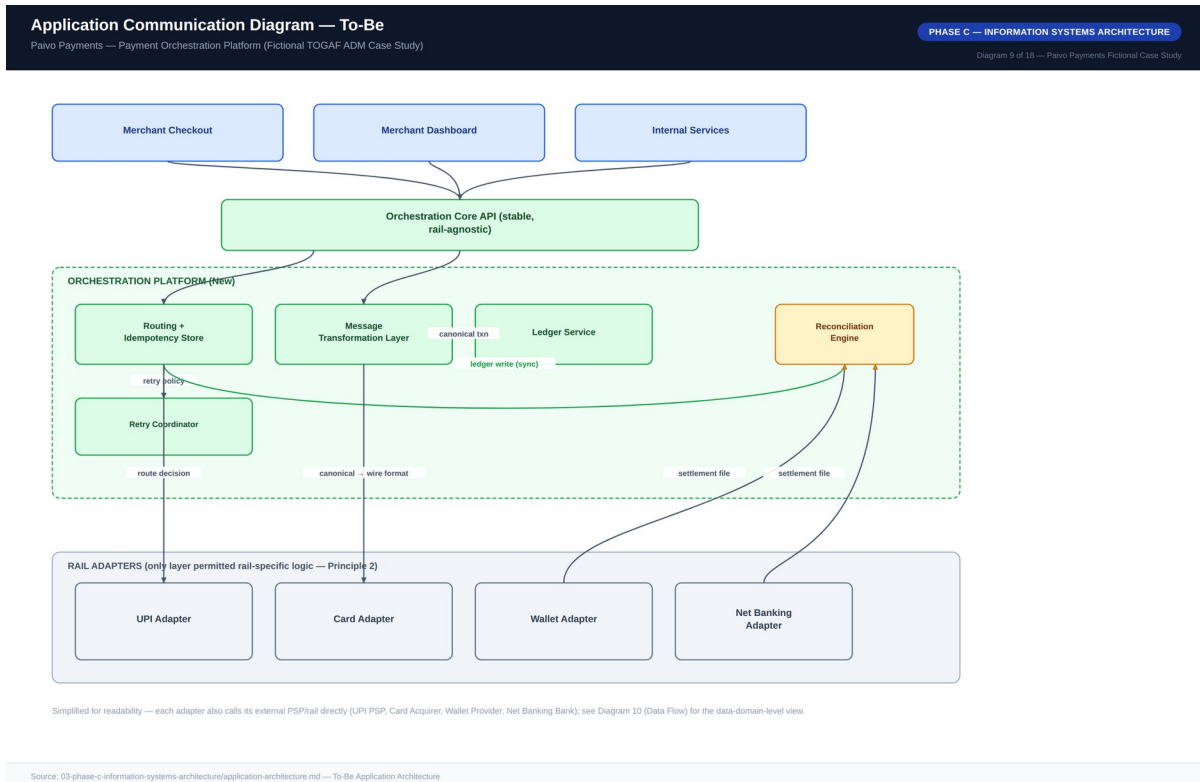
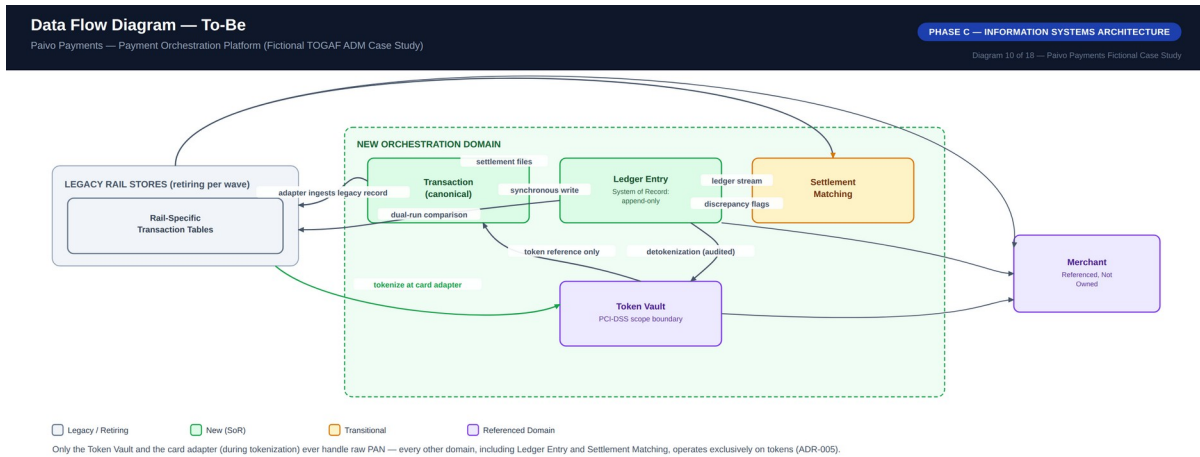


Diagram 9 — Application Communication Diagram, To-Be

5.2 Data Architecture

Data today is organized by rail integration, not by business entity: each of the ~20 point-to-point integrations owns its own transaction table, its own copy of merchant configuration, and its own settlement/reconciliation working store, typically a mix of relational tables and CSV settlement files parsed into spreadsheets. There is no canonical definition of a "transaction" that holds across rails — field names, status enumerations, and even currency/amount representations differ integration to integration. This produces four recurring problems the target architecture eliminates: no canonical transaction entity, no single settlement/ledger record, sensitive data sprawl (raw PAN and account data persisted across multiple integration-specific stores, the direct driver of the 11-system PCI-DSS audit scope), and no merchant-rail-agnostic settlement history.

The target state establishes four canonical data domains — Transaction, Ledger Entry, Merchant, and Settlement — owned centrally by the orchestration platform and consumed, not duplicated, by every other system. Sensitive cardholder and account data is tokenized at the adapter boundary (principle 5): the canonical Transaction entity carries tokens, never raw PAN, and only the PCI-scoped vault service can resolve a token back to raw data, and only for the narrow set of operations that require it. Data moves via an event-driven publish pattern — each adapter publishes a normalized transaction event to the orchestration core, the only writer to the Ledger Entry domain — rather than a shared-database pattern, which was assessed to violate principle 2 by letting adapter-specific write patterns leak into the core's data model, and to make independent adapter deployment unsafe.



Source: 03-phase-c-information-systems-architecture/data-architecture.md — Data Flow To-Be

Diagram 10 — Data Flow Diagram, To-Be

5.3 Data Ownership / Domains

Transaction and Ledger Entry are newly owned by the orchestration platform — Ledger Entry as the immutable, append-only record and single ledger of truth (ADR-002), Transaction as the canonical representation every adapter translates into at ingestion. Merchant identity and rail entitlements remain referenced, not owned, by the orchestration core, since KYC/onboarding stays out of scope. Settlement is the normalized representation of PSP settlement files, ingested by the reconciliation engine and matched against Ledger Entry records to produce discrepancy signals. The reconciliation engine consumes both streams as independent, read-only inputs and never writes back into either domain, preserving the ledger's immutability guarantee.

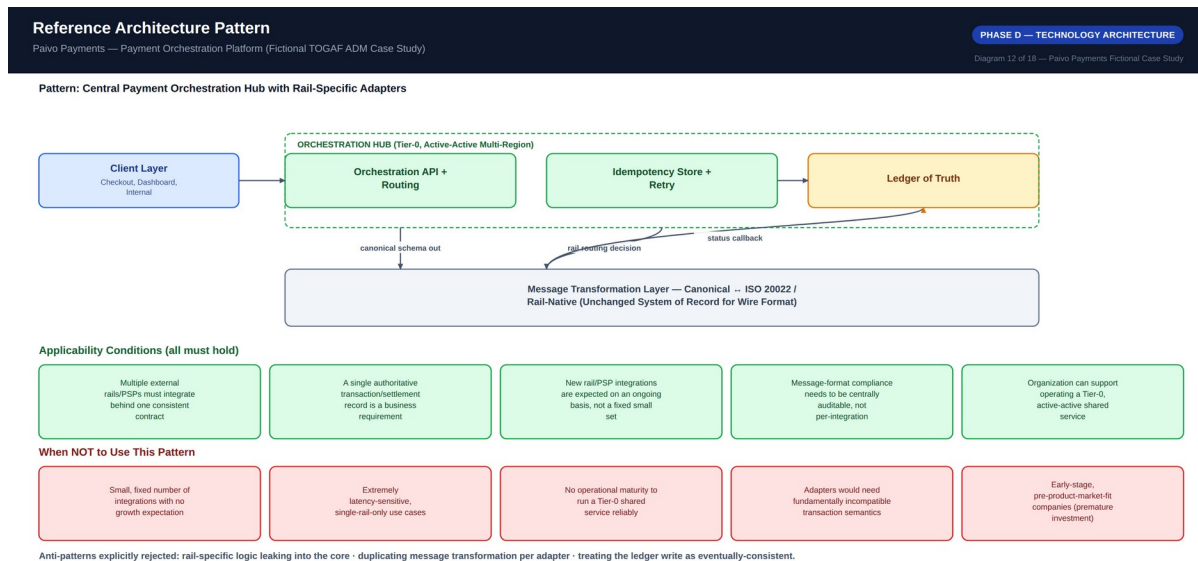
Fictional TOGAF ADM Case Study | Page 19 of 33

6. Phase D — Technology Architecture

6.1 Reference Architecture Pattern

The core technology pattern is a central orchestration hub (Tier-0, active-active multi-region) fronting rail-specific adapters (Tier-1, per-rail deployable) through a shared message transformation layer. The pattern applies when multiple external rails or PSPs must integrate behind a consistent internal contract, a single authoritative transaction record is a business requirement, new integrations are expected on an ongoing basis rather than a fixed small set, message format compliance needs to be centrally auditable, and the organization can support operating a Tier-0 shared service. Paivo's own situation — fourteen live rails, six/year growth, a regulatory mandate — clears this bar comfortably.

The ARB explicitly documented when a different topology should be chosen instead, to prevent this reference architecture being cargo-culted onto problems it doesn't fit: a small, fixed number of integrations with no growth expectation; extremely latency-sensitive, single-rail-only use cases where the extra hop through a general-purpose hub is pure overhead; organizations without the operational maturity to run a Tier-0 shared service, where the hub would become a worse single point of failure than the fragmented model it replaced; adapters that would need fundamentally incompatible transaction semantics, better served by a separate bounded context; and early-stage, pre-product-market-fit companies, where building this hub speculatively contradicts principle 7. Anti-patterns explicitly called out include leaking rail-specific logic into the orchestration core, duplicating message transformation per adapter for speed, and treating the hub as eventually-consistent-acceptable — the ledger write must be synchronous and part of a transaction's terminal state determination, or the single-ledger-of-truth guarantee is undermined.



Source: 04-phase-d-technology-architecture/reference-architecture.md

Diagram 12 — Reference Architecture Pattern

6.2 Technology Standards

The approved stack spans platform and infrastructure standards (managed Kubernetes-based container orchestration for multi-region active-active deployment; managed Kafka-compatible event streaming for adapter-to-core events and ledger event sourcing; a managed relational database with strict consistency mode for ledger persistence, since eventual-consistency stores were evaluated and rejected for that store specifically; a managed columnar/analytical store for reconciliation; and a managed low-latency key-value store with TTL for the idempotency key store, meeting the sub-10ms lookup requirement on the payment initiation hot path), identity, security and integration standards (managed KMS/secrets management with no application-managed secrets permitted; a certified third-party tokenization service or PCI-DSS Level 1 in-house vault for the token vault; a language-agnostic transformation runtime with an independently certified ISO 20022 schema validation library; and mandatory distributed tracing on every cross-service call in the orchestration path), and CI/CD standards (a mandatory, merge-blocking contract-testing gate for adapter interfaces, not advisory). Exceptions require an Architecture Contract stating which standard is deviated from, the quantified trade-off, and a time-boxed review date (default two quarters); an exception never revisited past its review date is treated as a governance gap and escalated to the Executive Sponsor. One notable exception is currently active and re-approved as a permanent catalog addition: the reconciliation engine uses a specialized time-series-optimized store rather than the standard analytical store, because its continuous, low-latency stream-join workload was a poor fit for a batch-oriented analytical store.

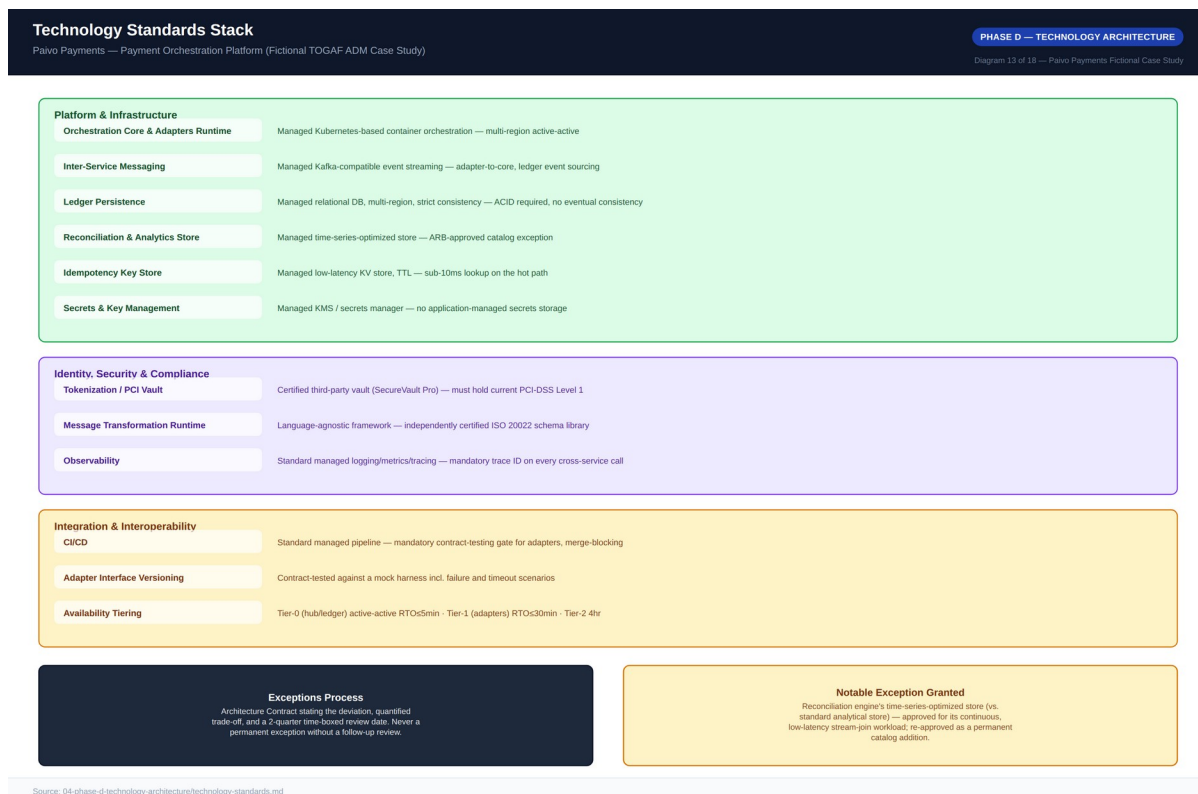


Diagram 13 — Technology Standards Stack

7. Phase E — Opportunities & Solutions

7.1 Gap Analysis

Twelve gaps were registered and scored 1–5 on Regulatory Urgency, Business Impact, and Implementation Risk if Unaddressed, summed to a priority score out of 15 — a deliberately simple additive model the ARB judged sufficient for a gap list this size after a side-by-side test against a more elaborate weighted model produced the same relative ordering. The top three gaps — no ISO 20022 structured messaging capability (14/15), no single ledger of truth (13/15), and inconsistent/absent idempotency enforcement (11/15, tied with PCI-DSS scope and fragmented reconciliation) — collectively define the Wave 0 and Wave 1 scope of the migration roadmap. Notably, new rail onboarding cost, despite being a headline driver of the business case, scores lower (8/15) than the ledger and idempotency gaps, because it is a business-impact-and-cost problem rather than a regulatory-deadline-and-risk problem; it is still addressed, but follows naturally from building the orchestration core and adapter contract correctly. Cross-border rail support scores lowest (4/15) and is intentionally not addressed by this program at all, corroborating that deferring it was the right sequencing call.

7.2 Solution Building Blocks

Each target capability decomposes into a discrete solution building block marked Build or Buy per architecture principle 3-equivalent reasoning (principle 11: prefer managed infrastructure for non-differentiating capability). The Orchestration Core and Ledger Service are built in-house — core differentiators with no commercial product matching Paivo's specific routing, idempotency, and audit-control requirements without heavy customization. The Message Transformation Layer buys a certified ISO 20022 schema validation library and builds the transformation orchestration logic around it. Rail Adapters are built against a standard adapter interface. The Token Vault buys a certified third-party tokenization service. The Reconciliation Engine and Compliance Reporting Interface are built in-house, since commercial reconciliation tools evaluated did not clear the bar for Paivo's specific settlement file variety. The Idempotency Key Store buys managed key-value infrastructure and builds the idempotency semantics on top.

Every rail adapter, regardless of rail, must implement the same contract to the orchestration core: ``initiate(canonical_transaction) -> adapter_reference``, ``status(adapter_reference) -> canonical_status``, and ``settlement_file() -> normalized_settlement_records``, plus mandatory honoring of the orchestration core's idempotency key with a guarantee against double-submission even across an adapter process restart. Adapters are contract-tested in CI against a mock harness exercising the full interface, including failure and timeout scenarios, before being allowed into the deployment pipeline — the mechanism that makes principle 7's "bounded, well-defined onboarding task" operational rather than aspirational. The Orchestration Core, Ledger Service, and Idempotency Key Store are built first since every other building block depends on them; the Message Transformation Layer follows immediately given the regulatory deadline; rail adapters are then built in migration-wave order, starting with the lowest-volume, lowest-risk rail.

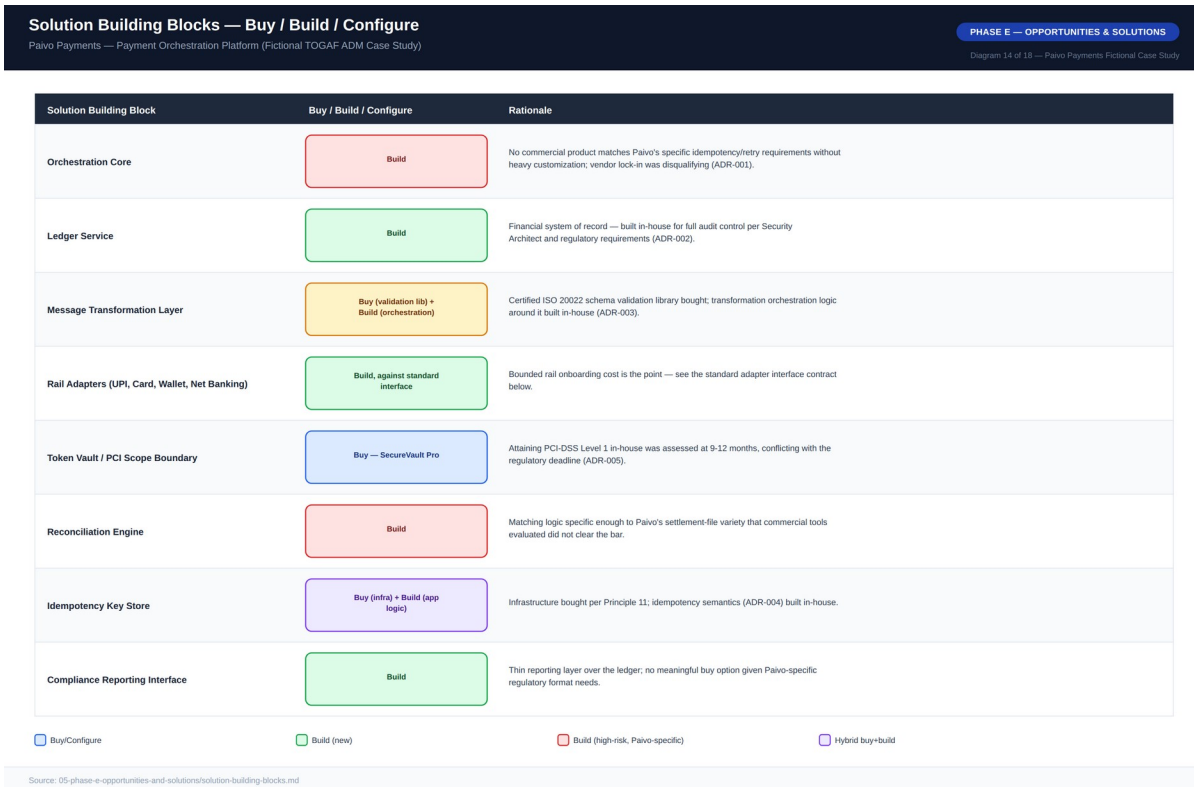


Diagram 14 — Solution Building Blocks, Buy / Build

7.3 Vendor Evaluation

Two separate buy decisions required vendor evaluation. For the orchestration core itself, build in-house won with a weighted score of 7.90/10 against Vendor A "Meridian Orchestrate" (6.30, runner-up), Vendor B "FlowGate Platform" (5.80), and Vendor C "PayMesh Connect" (5.45) — winning on the two criteria weighted heaviest: fit to Paivo's idempotency/retry requirements (20%) and vendor lock-in/substitutability (15%), since every commercial option would have made the orchestration core — the platform's most architecturally central component — dependent on a single vendor's roadmap, directly violating principle 9. Meridian Orchestrate was the strongest commercial runner-up on time-to-first-production-rail and ISO 20022 extensibility, but its proprietary routing and idempotency logic is not portable, reproducing exactly the lock-in risk principle 9 exists to avoid. The accepted trade-off of building in-house is a longer time to first production rail (5/10 vs. 7-8/10 for vendor options), managed by front-loading orchestration core development in Wave 0.

For PCI-DSS tokenization specifically, the calculus reversed: Vendor D "SecureVault Pro" was selected (7.95/10) over Vendor E "TokenShield" (7.0/10, runner-up) and an in-house PCI-DSS Level 1 vault (6.2/10). SecureVault Pro won primarily on detokenization latency — an estimated 12ms versus TokenShield's 35ms added to the card adapter's hot-path calls. Building in-house was rejected principally on timeline: achieving Level 1 certification independently was assessed at 9-12 months, directly conflicting with the regulatory deadline, and secondarily on cost, with first-year certification estimated around \$650K plus recurring annual assessment that a vendor amortizes across its customer base. The accepted trade-off is an external dependency for detokenization latency, which dispute-handling and reconciliation workflows must design around via time-boxed cached access rather than repeated vault calls (see ADR-005).

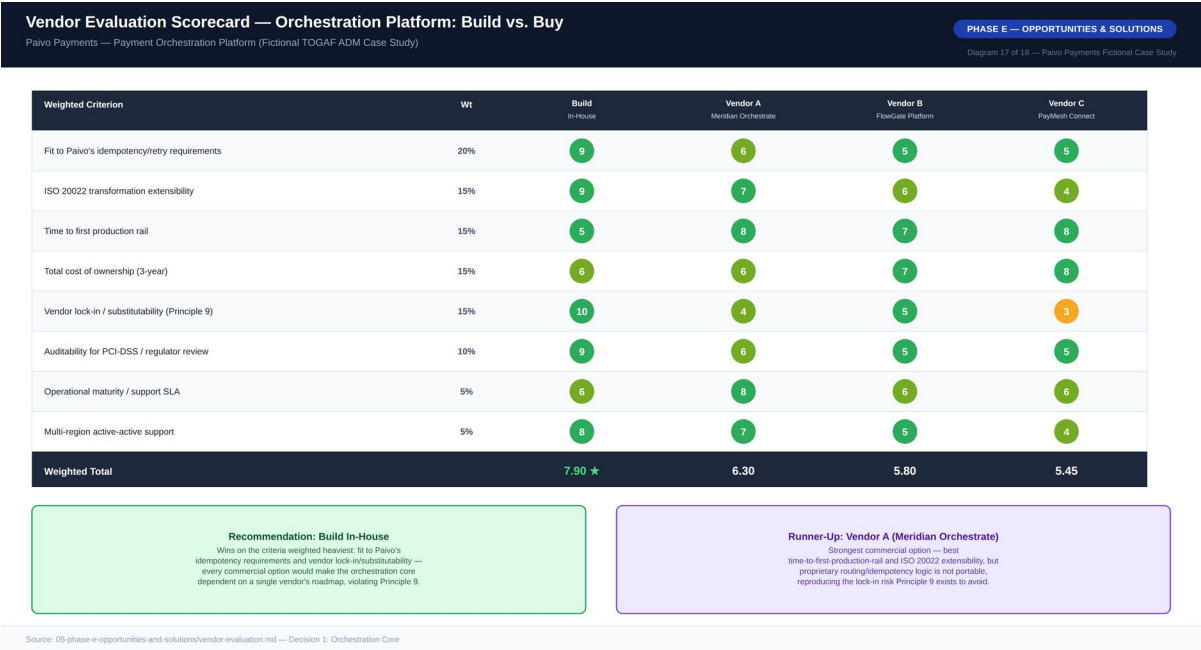


Diagram 17 — Vendor Evaluation Scorecard, Orchestration Core: Build vs. Buy

8. Phase F — Migration Planning

8.1 Migration Roadmap

The roadmap runs six waves. Wave 0 (Foundation, Q3–Q4 2026) builds the Orchestration Core, Ledger Service, Idempotency Store, and Message Transformation Layer with no live rail traffic, exiting when core services pass load and chaos testing at Tier-0 SLA and ISO 20022 schema validation passes the full compatibility suite. Wave 1 (Q1 2027) migrates net banking — the lowest transaction volume, single dominant PSP — as the first live adapter in dual-run against the legacy integration, exiting after four weeks of dual-run with zero ledger/legacy discrepancy beyond tolerance and a proven rollback. Waves 2 through 4 migrate wallets (2 PSPs), UPI (highest transaction count), and cards (activating the full PCI-DSS scope boundary via the token vault, exiting only once an external QSA confirms reduced audit scope) respectively, through Q4 2027. Wave 5 (Q4 2027–Q1 2028) decommissions remaining legacy integrations and completes full ISO 20022 compliance certification.

An alternative sequencing — highest-volume rail first, to prove value fastest to sales and leadership — was considered and rejected: it would front-load risk onto the rail Paivo can least afford to destabilize, for a marketing benefit the ARB judged not worth the exposure. Every cutover is a single mechanical event, not a milestone label: the Routing Engine holds a per-rail routing configuration state (`legacy` or `migrated`), and cutover is that flag flipping, executable in minutes rather than on a redeploy cycle. During the dual-run window, live traffic is routed end-to-end through the legacy integration while a shadow copy simultaneously flows through the new adapter, transformation layer, and ledger, with outputs compared transaction-by-transaction — dark-launch, not A/B, so a defect found during dual-run has zero merchant or customer impact. Rollback is the same flag flip in reverse, triggered by a post-cutover discrepancy exceeding tolerance, a Tier-0 SLA breach, or an ARB-declared incident; ledger entries already written are never deleted or mutated, per the ledger's append-only design, and a rolled-back rail re-enters dual-run rather than attempting a second immediate cutover.

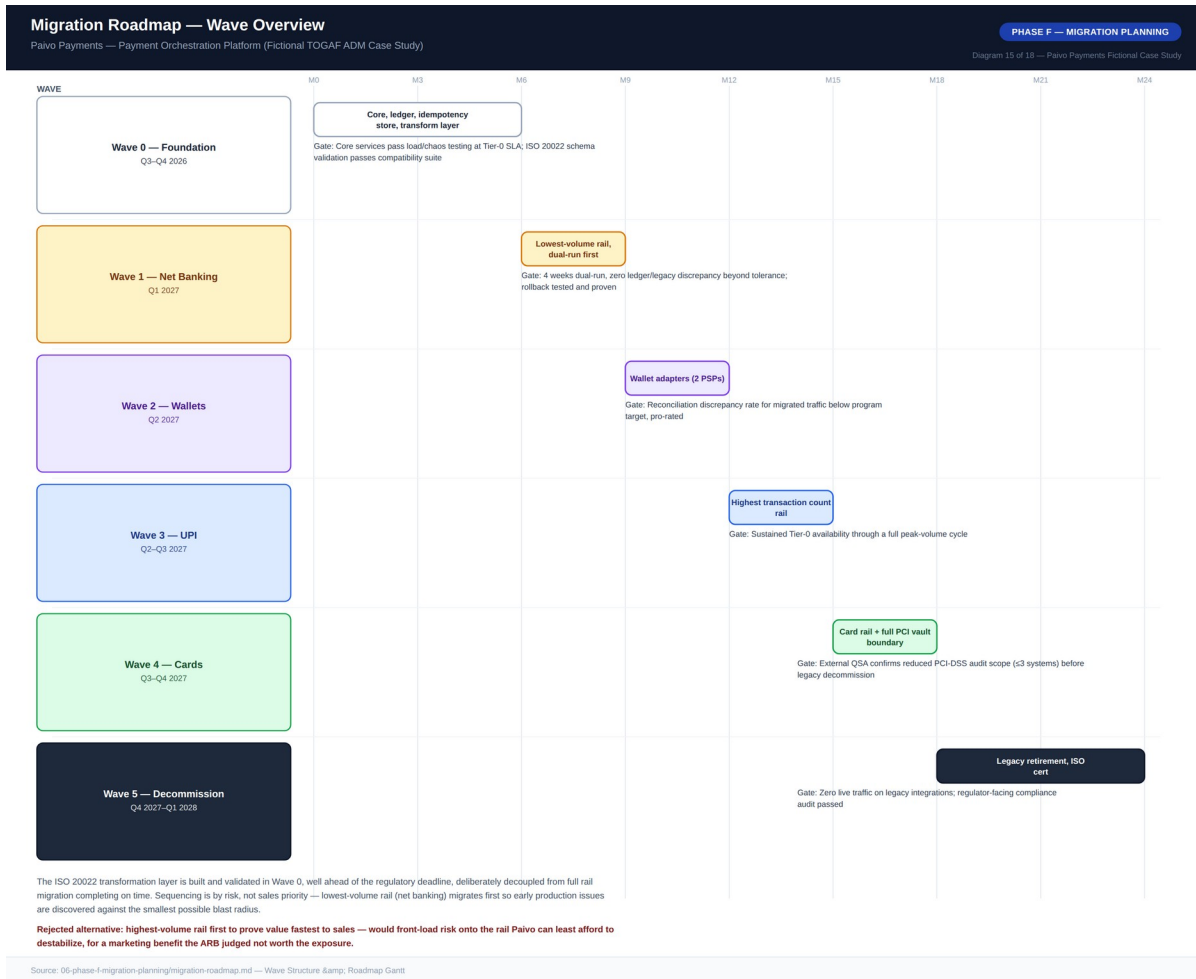


Diagram 15 — Migration Roadmap, Wave Overview

8.2 Transition Architecture — Dual-Rail Coexistence

During Waves 1–4 (roughly Q1 2027–Q4 2027), Paivo operates a defined intermediate state, Dual-Rail Coexistence: legacy point-to-point integrations continue running exactly as in the as-is architecture for rails not yet migrated, including their independent reconciliation pipelines, while the orchestration hub, ledger, and transformation layer actively process traffic for migrated rails. A reconciliation bridge process consolidates ledger data from migrated rails with legacy reconciliation output from not-yet-migrated rails into a single finance-facing view, so Finance Operations does not have to context-switch between two entirely separate reporting processes. A big-bang cutover migrating all rails simultaneously was evaluated in Phase A and rejected specifically because it would require validating the entire platform’s correctness across every rail’s edge cases at once, with no ability to isolate a defect to a single rail. Counterintuitively, mid-migration Paivo runs more systems than either the as-is or to-be state — flagged to the Board explicitly as a transitional Year-1 cost, not runaway scope.

A second transition state, Reconciliation Bridge Retirement, spans Wave 5 (Q4 2027–Q1 2028): once all rails have migrated, the reconciliation bridge keeps running for a defined wind-down period rather than being removed the moment the last rail cuts over, so Finance Operations’ month-end close gets at least one full closed period fully on the new unified ledger before the bridge — and the manual reconciliation muscle

memory it represents — is retired. Retiring the bridge in the same release as the last cutover was considered and rejected: it would remove the fallback safety net at precisely the moment the new ledger handles 100% of volume for the first time, the highest-risk moment to also remove the fallback.

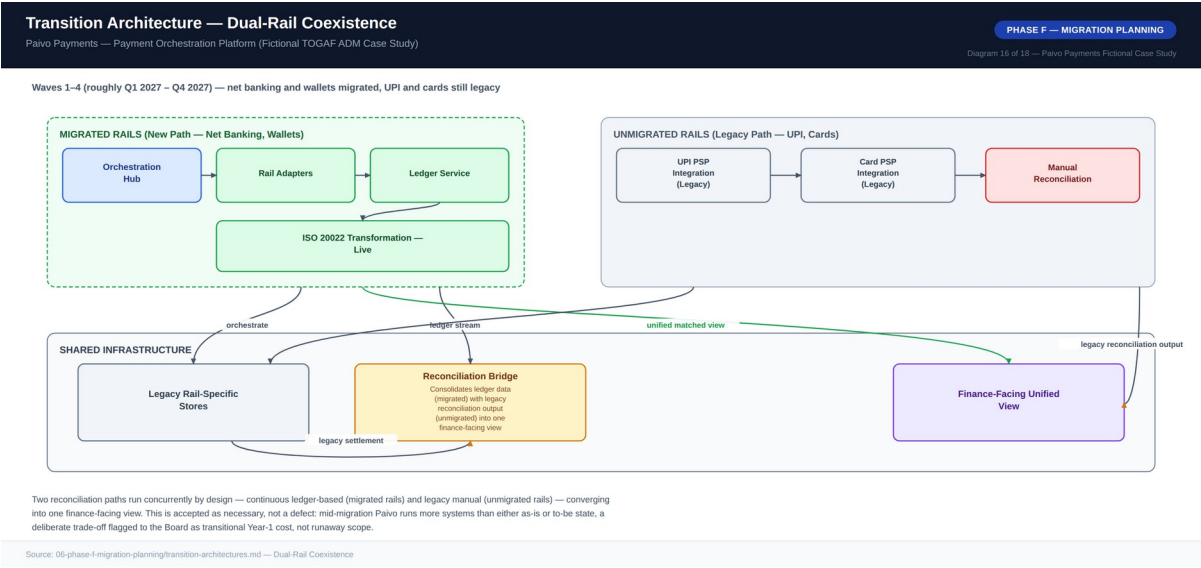


Diagram 16 — Transition Architecture, Dual-Rail Coexistence

9. Phase G — Implementation Governance

Phase G governs how the target architecture defined in Phases B–E is actually built, ensuring delivery teams stay within the approved architecture without turning the ARB into a bottleneck for routine engineering. Six compliance checkpoints apply across the delivery lifecycle: Architecture Contract approval (ARB, before build starts on any new solution building block or interface); the adapter contract test gate (Platform Engineering tooling, ARB-defined, merge-blocking on every CI run); PCI-DSS scope review (Security Architect and Head of Risk & Compliance, before any change touching cardholder data flow); wave go/no-go review (ARB, before each migration wave cutover); ISO 20022 compliance validation (Head of Risk & Compliance with external auditor liaison, before each rail's transformation logic goes live and at each schema version update); and a quarterly architecture health audit (Chief Architect, principle compliance across all shipped building blocks).

9.1 Non-Compliance Handling

Non-compliance is calibrated into three tiers so governance responds proportionally rather than treating every deviation as a crisis. Tier 1, unintentional drift, is handled through a contract amendment reviewed at the next standing ARB session, with no escalation beyond normal process. Tier 2, an undisclosed deviation discovered after the fact — for example, a quarterly health audit finding rail-specific logic leaking into the orchestration core, undocumented — triggers a formal non-compliance finding, a mandatory remediation plan with a committed date, and a follow-up audit, with the Head of Platform Engineering accountable for execution. Tier 3, a deviation creating active regulatory or security risk — for example, cardholder data found flowing into an out-of-scope system — triggers immediate escalation to the Executive Sponsor and Head of Risk & Compliance, paused work on the affected component, and a mandatory root-cause review at the next ARB session. To date, the program has recorded two Tier 1 amendments, both related to the reconciliation engine's data store exception, and zero Tier 2 or Tier 3 findings — attributed largely to the mandatory principle-impact section in the Architecture Contract template catching issues before build rather than after.

9.2 Architecture Contracts

An Architecture Contract is the formal agreement between the ARB and a delivery team, structured across nine mandatory sections: requesting team and business owner; traceability to a stated Phase A requirement and Phase E gap analysis entry (a contract that cannot cite one is returned unreviewed); scope and boundary of the solution building block; a principle impact statement; interface commitments for any multi-consumer component; non-functional requirements (availability tier, latency budget, data classification); acceptance criteria; the deviation and amendment process; and the approval record. The worked example — the UPI Rail Adapter contract, submitted 2027-04-02 and reviewed 2027-04-09 — traces to the gap analysis entry "new rail onboarding cost is high and unpredictable" (priority score 8) and Wave 3 of the roadmap, sets a latency budget of 800ms p99 for the `initiate()` call (against UPI's own 600ms PSP SLA, with 200ms reserved for transformation layer overhead), and was approved unanimously with one condition: the dual-run discrepancy monitoring dashboard had to be reviewed by Finance Operations before cutover sign-off, added at the Finance representative's request during review.

10. Phase H — Change Management

An architecture that is technically correct but organizationally unadopted delivers none of the business case's projected value — the reconciliation-labor and rail-onboarding savings depend entirely on Finance Operations, Platform Engineering, and merchant-facing teams actually using the new capabilities as designed. This plan treats organizational adoption as a first-class deliverable with the same governance rigor as the technical architecture.

Stakeholder Group	Nature of Change	Impact Level
Finance Operations (4.5 FTE)	Moves from manual, rail-fragmented reconciliation to operating a continuous automated matching engine; role shifts from data assembly to exception investigation	High
Legacy Integration Engineers	Integrations retired progressively; individuals transition to adapter-building roles or other engineering priorities	High
Platform Engineering	New standard interface, contract-testing discipline, and ARB governance to work within — more structured than prior ad hoc builds	Medium
Merchant-facing Account Management	New 2–3 week rail onboarding (vs. 10–14) becomes a sales asset; needs updated talking points during the transitional window	Medium
Security & Compliance	New centralized PCI-DSS scope boundary changes what they audit — less breadth, more depth on fewer systems	Medium
Merchants (external)	No process change expected if migration executes as planned; communication is precautionary, not because disruption is expected	Low

Training is wave-aligned: Finance Operations gets a structured three-week transition curriculum per wave, run in parallel with that wave's dual-run period, learning the new ledger-based matching tooling on live shadow data before it becomes their primary system of record. Legacy Integration Engineers are offered a defined path onto adapter-building teams with structured onboarding to the standard adapter interface, explicitly framed as a skills transition — their deep PSP-specific knowledge is directly valuable in adapter migration work — rather than a redundancy process, plus two weekly "migration office hours" sessions during Waves 1–4. Adoption metrics reviewed alongside technical wave-gate criteria include 100% of reconciliation work for migrated rails performed via new tooling within two weeks of cutover (with legacy tooling access revoked at that point), ≥90% of Architecture Contract submissions using the mandatory template correctly by Wave 3, ≥80% of affected Legacy Integration Engineers placed into new roles within one quarter of decommission, merchant support tickets held to baseline during any wave's cutover week, and Finance month-end close duration reduced from ~6 business days to ≤3 by Wave 5. Merchant communication is deliberately conservative — triggered only by genuine customer-visible impact — because over-communicating a live-funds migration risks manufacturing concern about a system that, if the transition architecture works as designed, merchants should never notice changed at all.

11. Architecture Decision Records

Five ADRs anchor the architecturally significant decisions in this program. Each follows the standard Status / Context / Decision / Alternatives Considered / Consequences / Stakeholders format and is cross-referenced from the architecture contracts and technology standards that depend on it.

ADR-001: Central Orchestration Hub vs. Point-to-Point Mesh vs. Hybrid Shared-Ledger Mesh

Adopt a central orchestration hub with rail-specific adapters, all traffic routing through one core owning routing, idempotency, retries, and ledger writes. An improved point-to-point mesh with shared libraries was rejected for solving the symptom, not the root cause; a hybrid shared-ledger-with-decentralized-routing design (the most seriously contested alternative, ~35 fewer person-months) was rejected because it does not solve idempotency consistency. Accepted trade-off: the hub becomes the single most architecturally critical component, requiring active-active multi-region deployment and adding an estimated 20% to steady-state infrastructure cost. This decision produced the program's one recorded escalation to the Executive Sponsor, who ratified the ARB's recommendation citing the existing double-debit incident history as decisive.

ADR-002: Single Ledger of Truth — Synchronous Write, Immutable Append-Only Design

The ledger is append-only and immutable, written synchronously by the orchestration core as part of determining a transaction's terminal state; corrections are new linked adjustment entries, never mutations. An asynchronous write was rejected because it reintroduces an audit gap where a crash could leave a caller-visible terminal status with no ledger record. A mutable, update-in-place design was rejected for destroying the audit trail regulators and Finance require. Accepted trade-off: synchronous writes add a measured 18ms p99 to the orchestration core's hot path, absorbed by design in adapter latency budgets, and storage grows unboundedly, requiring a defined archival/cold-storage tiering policy.

ADR-003: ISO 20022 Transformation Layer Placement — Shared Layer vs. Per-Adapter vs. Embedded in Core

Build a single, shared message transformation layer as its own independently deployable, versioned service between the orchestration core and adapters. Per-adapter "fat adapter" transformation was rejected for reproducing, one layer down, the point-to-point maintenance burden the program exists to eliminate. Embedding transformation in the core was rejected for violating principle 2 by coupling the core's release cycle to externally-driven format changes. Accepted trade-off: an added network hop measured at approximately 8–12ms per transaction in staging, and an external interface dependency for adapter teams, mitigated by strict versioning and a contract-testing gate.

ADR-004: Idempotency and Retry Strategy for Cross-Rail Transactions

Every state-changing orchestration API requires a caller-supplied idempotency key, validated and stored centrally in a dedicated key-value store with TTL; adapters additionally implement rail-side de-duplication as defense in depth, but the platform-level guarantee does not depend on adapters getting this right. Idempotency enforced only at the adapter level was rejected as the exact pattern that produced the current double-debit incidents. A database unique-constraint approach was rejected for creating false-positive risk when a merchant legitimately submits two transactions of identical amount and near-identical timing. Accepted trade-off: every caller must correctly generate and consistently reuse idempotency keys, and the key store itself becomes a Tier-0-adjacent dependency requiring the same multi-region resilience as the orchestration core.

ADR-005: PCI-DSS Scope Isolation Strategy

Tokenize cardholder data at the card rail adapter — the earliest point it enters Paivo's systems — using a certified third-party tokenization vault (Vendor D, SecureVault Pro); every downstream system operates exclusively on tokens. Tokenizing at the orchestration core instead was rejected because it would require raw PAN to transit through to the core first, expanding scope to include the core itself. Building an in-house Level 1 certified vault was rejected primarily on the 9–12 month certification timeline conflicting with the regulatory deadline, and secondarily on the ~\$650K illustrative first-year cost. Accepted trade-off: an external vendor dependency for detokenization latency (12ms per call) that dispute-handling and other workflows must design around via time-boxed cached access rather than repeated calls.

Appendix A — Diagram Catalog Index

The full reference diagram catalog for this program comprises 18 diagrams spanning the Preliminary Phase through Phase F, delivered as high-resolution PNG files alongside this document.

No.	Diagram Title	ADM Phase
01	Architecture Vision One-Pager	Phase A
02	Stakeholder Power / Interest Map	Phase A
03	Architecture Governance Organization	Preliminary
04	Business Capability Map, Maturity Heat Map	Phase B
05	Business Process Flow, As-Is (Payment Initiation to Reconciliation)	Phase B
06	Business Process Flow, To-Be (Payment Initiation to Reconciliation)	Phase B
07	Business Footprint Diagram	Phase B
08	Application Communication Diagram, As-Is	Phase C
09	Application Communication Diagram, To-Be	Phase C
10	Data Flow Diagram, To-Be	Phase C
11	Data Ownership / Domain Diagram	Phase C
12	Reference Architecture Pattern	Phase D
13	Technology Standards Stack	Phase D
14	Solution Building Blocks, Buy / Build	Phase E
15	Migration Roadmap, Wave Overview	Phase F
16	Transition Architecture, Dual-Rail Coexistence	Phase F
17	Vendor Evaluation Scorecard	Phase E
18	Business Case Financial Summary	Phase A

Appendix B — Disclaimer

This entire document — Paivo Payments, all named individuals and roles, all vendor names (Meridian Orchestrate, FlowGate Platform, PayMesh Connect, SecureVault Pro, TokenShield), and every dollar figure, metric, and date — is a fictional case study constructed for architecture-practice demonstration purposes. Figures are designed to be internally consistent and arithmetically sound so that the reasoning pattern is transparent, not to represent any real payments company's actual data. No inference should be drawn about any real company, product, or individual.